- **Array Processor:**

An array processor is a processor that performs computations on large arrays of data. The two different types of array processors are:

1. Attached Array Processor
2. SIMD Array Processor

Both types of array processors manipulate vectors, but their internal organisation is different.

Note: A vector is an ordered set (1-D array) of data items.

1. Attached Array Processor:

An attached array processor is an auxiliary processor attached to a general-purpose computer. Its purpose is to enhance the performance of the computer by providing vector processing for complex scientific applications, especially numerical computation. It achieves high performance by means of parallel processing with multiple functional units.

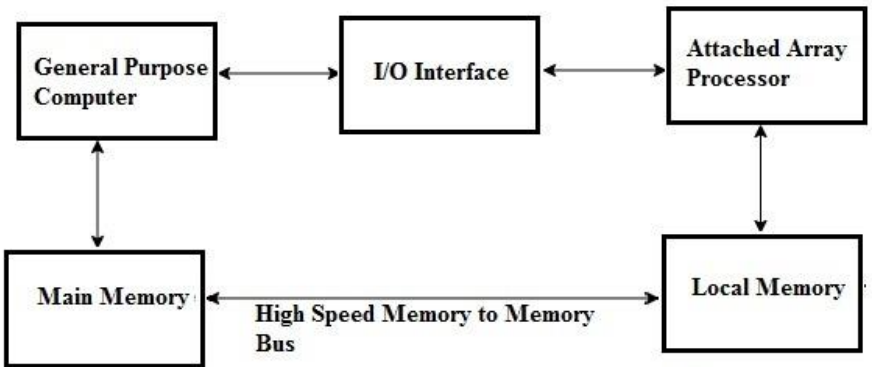


Fig. 1: Attached array processor with host computer.

The host computer is a general-purpose commercial machine, and the attached processor is a back-end machine driven by the host. It behaves like an external interface. The data for the attached processor is transferred from main memory to local memory via a high-speed bus.

2. SIMD Array Processor

A SIMD (Single Instruction Stream, Multiple Data Stream) array processor is a computer with multiple processing units operating in parallel. The processing units are synchronised to perform the same operation under the control of a common control unit.

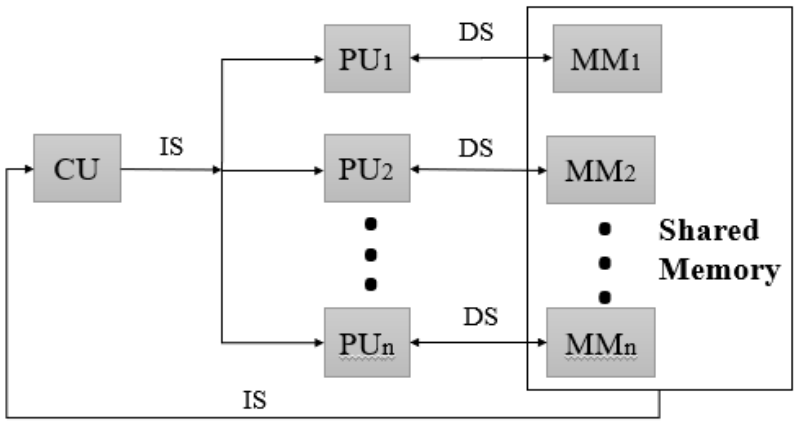


Fig. 2: SIMD array processor organization

It contains a set of identical Processing Elements (PEs), each having local memory M.

Each PE includes:

- An ALU (Arithmetic Logic Unit)
- Floating-point arithmetic unit
- Working registers

The control unit decodes the instructions and determines how the instructions are to be executed.

- Scalar and program-control instructions are executed directly within the CPU.
- Vector instructions are broadcast to all PEs simultaneously.

SIMD processors are best suited for numerical problems that can be expressed in vector or matrix form.

- **Vector Processor:**

A vector processor is a CPU designed to execute a single instruction across large, one-dimensional arrays of data (vectors) simultaneously.

Different Types of Vector Instructions:

1. Vector–Vector Instructions: As shown in Fig. 1, one or two vector operands are fetched from their respective vector registers, enter through a functional pipeline unit, and produce results in another vector register.

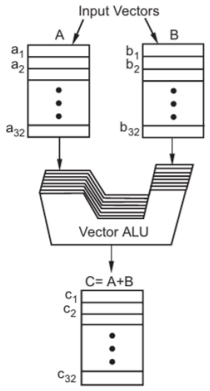


Fig. 1: Vector–Vector Instruction

These instructions are defined by the following mapping examples:

- $f_1: V_j \rightarrow V_i$ Example: $B(i) \leftarrow \sqrt{A(i)}$
- $f_2: V_A + V_B \rightarrow V_C$ Example: $V_C \leftarrow V_A + V_B$

2. Vector–Scalar Instructions: Fig. 2 shows a vector-scalar instruction corresponding to the following mapping: $f_3: S \times V_j \rightarrow V_i$

Each element of the vector V_K is multiplied by the scalar stored in the S register.

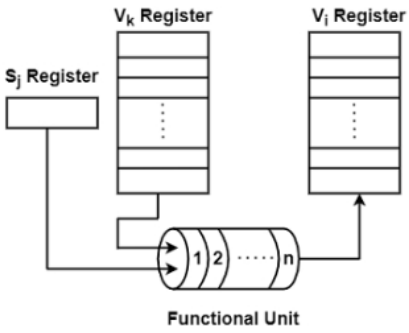


Fig. 2: Vector–Scalar Instruction

3. Vector–Memory Instructions: This corresponds to vector-load or vector-store operations (Fig. 3), performed element by element between the vector register (V) and memory (M).

The operations are:

Vector Load:

$$f_4: M \rightarrow V$$

Vector Store

$$f_5: V \rightarrow M$$

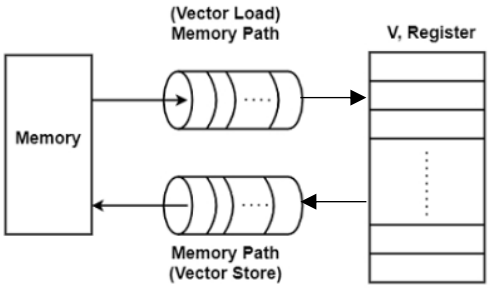


Fig. 3: Vector–Memory Instructions

Terminology:

1. Strip Mining (Vector Loops): When a vector has a length greater than that of the vector registers, segmentation of the long vector into fixed-length segments becomes necessary. This technique is called **strip mining**. One vector segment is processed at a time. For example, in the Cray computer, the vector segment length is 64 elements.

2. Vector Stride (Skew Distance): Vector stride refers to the fixed step size (in bytes or elements) between consecutive memory accesses when loading or storing vector data. It determines how elements are loaded from or stored in non-contiguous memory locations during vectorised computations.

A vector is a set of scalar data items of the same type stored in memory. Usually, vector elements are ordered with a fixed addressing increment between successive elements called the **stride**. Vector elements are not necessarily stored in contiguous memory locations. For example, matrix elements may be stored in row-major or column-major order. Each row, column, or diagonal of a matrix can be used as a vector.

- Row elements are stored in contiguous locations with a unit stride.
- Column elements are stored with a stride of n, where $n$is the matrix order.
- Diagonal elements have a stride of $n + 1$(for a square matrix).

Vector Gather and Vector Scatter Instructions:

In vector processors (such as SIMD architectures), gather and scatter instructions are used to efficiently handle non-contiguous memory access patterns.

- **Vector Gather Instructions:** A gather operation collects data from multiple memory locations (which are not necessarily contiguous) and loads them into a vector register. It is used when data elements are stored at arbitrary locations in memory.

Example: Memory array: A=[10,20,30,40,50,60,70,80]
Indices to gather: [2,5,7]
Gather operation: V=[A[2],A[5],A[7]]
Result: V=[30,60,80]
Instruction Format: VGATHER V1, (R1 + V2)
Meaning: Gather values from the base address stored in register R1 using the indices contained in vector register V2.

- **Vector Scatter Instructions** A scatter operation stores vector elements into non-contiguous memory locations based on an index vector. It is used when writing data to arbitrary locations in memory. Useful for Sparse data storage, database operations, and parallel computing

Example: Values to store: V=[100,200,400]

Indices: [1,4,6]

Instruction Format: VSCATTER (R1 + V2), V1

Meaning: Scatter values from vector register V1 into memory locations indexed by vector register V2. The base address of the array is stored in register R1.